

# 00a. はじめての方へ（超入門） — パソコン操作の基礎から

---

この章は、「**プログラミングはおろか、パソコンの操作にもあまり自信がない**」という方のための、いちばん最初の一步です。ここで出てくる言葉は、後の章ですっと使います。急がず、知っている所は飛ばしながら読んでください。

この章のゴール：「フォルダ」「パス」「ターミナル」「コマンド」という4つの言葉に、こわさを感じなくなること。これだけで十分です。

## 1. まず深呼吸 — これから何を学ぶの？

---

あなたがこれから引き継ぐのは、「**業務工数システム**」という**Webアプリ**を作り・動かし・直し続ける仕事です。

最初に知っておいてほしいことが3つあります。

1. **最初は意味が分からなくて当たり前** です。専門用語が次々出てきますが、全部を一度に理解する必要はありません。「そういう名前のものがあるんだな」で先に進んでOKです。
2. **手を動かすと分かってきます**。読むだけより、実際にコマンドを打ち、画面を触ると、ぐっと理解が進みます。
3. **壊しても大丈夫な場所（自分のPCの中）で練習します**。本番（みんなが使っている環境）にいきなり触ることはありません。安心してください。

## 2. パソコンの中の「住所」を理解する：ファイル・フォルダ・パス

---

プログラムは、たくさんの **ファイル** からできています。まずファイルとフォルダの関係を整理しましょう。

## ファイルとは

データの「1つのまとまり」。文書・画像・プログラムなど。名前が付いています。

- 例： `Login.tsx`、`models.py`、`data.xlsx`

## 拡張子（かくちょうし）とは

ファイル名の最後、ドット（`.`）以降の部分。**そのファイルの種類**を表します。

- `.txt` → ふつうの文字ファイル
- `.py` → Python（パイソン）のプログラム
- `.tsx` / `.ts` → TypeScript（タイプスクリプト）のプログラム
- `.css` → 見た目を決めるファイル
- `.md` → マークダウン（この教育資料の形式）
- `.xlsx` → Excelファイル

Windowsは初期設定で拡張子を隠していることがあります。エクスプローラー（フォルダを見る画面）の「表示」メニューで「ファイル名拡張子」にチェックを入れる则表示されます。プログラムを扱うときは表示しておくのがおすすめです。

## フォルダ（ディレクトリ）とは

ファイルを整理して入れておく「箱」。箱の中にさらに箱（フォルダ）を入れられます。

- 「フォルダ」と「ディレクトリ」は **同じ意味** です。プログラムの世界では「ディレクトリ」と呼ぶことが多いです。

```
hozen-kosu-React    ← いちばん外側のフォルダ（このプロジェクト全体）
├─ frontend          ← フォルダ（画面側のコード一式）
│   └─ src           ← フォルダ
│       └─ MainPage  ← フォルダ
│           └─ Login.tsx ← ファイル（ログイン画面のコード）
├─ kosu              ← フォルダ（裏側の処理一式）
│   └─ models.py     ← ファイル
└─ manage.py         ← ファイル
```

## パス (path) とは — ファイルの「住所」

あるファイルが「どのフォルダの、どのフォルダの中にあるか」を表した文字列を **パス** と言います。住所のようなものです。

- Windowsでは、フォルダの区切りに **\** (**円マーク／バックスラッシュ**) を使います。
  - 例: `C:\Users\あなた\Documents\git-projects\hozen-kosu-React\manage.py`
- プログラムの世界 (このアプリのコードの中) では **/** (**スラッシュ**) を使うことも多いです。
  - 例: `frontend/src/MainPage/Login.tsx`
- どちらも「フォルダをたどっていく道順」を表している、と理解すればOKです。

### 絶対パスと相対パス

- **絶対パス**: `c:\` から始まる「完全な住所」。どこから見ても同じ場所を指す。
- **相対パス**: 「今いる場所から見た道順」。`./` は「今いるフォルダ」、`../` は「1つ上のフォルダ」を表す。後の章でコードを読むときに `../api/axios` のような書き方が出てきますが、これは相対パスです。

## 3. ターミナル (PowerShell) — 文字でPCに命令する黒い画面

ふだんは、アイコンをクリックしてアプリを動かしますよね。プログラミングでは、それに加えて **文字で命令を打ち込む** 操作をよく使います。そのための画面を **ターミナル** と呼びます。

Windowsの標準ターミナルが **PowerShell (パワーシェル)** です。

### PowerShellの開き方

1. キーボードの **Windowsキー** を押す (または画面左下のスタートボタン)。
2. `PowerShell` と入力する。
3. 出てきた「Windows PowerShell」をクリック。
4. 黒い (または青い) 画面が開きます。これがターミナルです。

開くと、こんな表示が出ます：

```
PS C:\Users\あなた>
```

- `PS` は PowerShell の印。
- `C:\Users\あなた` は **今いるフォルダ（カレントディレクトリ）** = 「今あなたが立っている場所」。
- `>` の右側に、命令（コマンド）を打ち込みます。

---

## 4. コマンド — 命令の言葉

---

ターミナルに打ち込む命令を **コマンド** と言います。コマンドは「**コマンド名 + 追加の指定**」という形が基本です。

```
cd Documents
```

- `cd` がコマンド名（change directory = フォルダを移動せよ、の意味）。
- `Documents` が「どこへ移動するか」の指定。
- 打ち込んだら、最後に **Enterキー** を押して実行します。

### 最初に覚えると便利なコマンド3つ

```
cd フォルダ名
```

今いる場所から、指定したフォルダへ **移動** する。 `cd ..` で1つ上のフォルダへ戻る。

```
ls
```

今いるフォルダの中身（ファイル・フォルダ）の **一覧を表示** する。「今どこに何があるか」を確認できる。

```
pwd
```

今いるフォルダの **住所（パス）を表示** する。「自分が今どこにいるか」が分かる。

## コマンドを打つときのコツ

- **コピー&貼り付けが使えます**：この資料のコマンドをコピーし、PowerShellの画面で **右クリック**（または `Ctrl + V`）すると貼り付けられます。打ち間違い防止に有効。
- **Tabキーで補完**：フォルダ名を途中まで打って **Tab** キーを押すと、続きを自動入力してくれます。
- **↑キーで前のコマンド**：上矢印キーで、前に打ったコマンドを呼び出せます。
- **大文字小文字・スペースに注意**：コマンドは正確に。とくに半角スペースの有無は意味が変わります。
- **エラーが出て壊れません**：間違ったコマンドを打っても、たいていは赤い文字で「できませんでした」と出るだけです。落ち着いて読み、打ち直しましょう。

「**コマンドが見つかりません**」と出たら？ そのコマンド（プログラム）がまだインストールされていない、または「PATH」に登録されていない、というサインです。 `01_環境構築.md` でインストールを進めると解決します。

## 5. インターネットとサーバーの超ざっくり理解

Webアプリを理解するうえで、最低限のイメージを持っておきましょう。

- あなたがブラウザ（Chrome等）でURLを開くと、ブラウザは **インターネットを通じて「サーバー」というコンピュータにお願い** を送ります。
- サーバーは、お願いに応じて画面やデータを **返事** します。
- ブラウザはその返事を受け取って、画面に表示します。

あなたのブラウザ —「この画面ください」→ サーバー  
(手元のPC)      ←「はい、どうぞ」———— (どこかにあるコンピュータ)

この「お願い」と「返事」のやり取りが、Webアプリのすべての基本です。くわしくは `00_導入と全体像.md` で説明します。今は「ブラウザとサーバーが会話している」とだけ覚えればOKです。

## 6. ブラウザの開発者ツール（F12） — 困ったときの味方

---

ブラウザには、画面の裏側を調べる機能が隠れています。キーボードの **F12キー** を押すと開きます（または画面で右クリック → 「検証」）。

- 最初は難しそうに見えますが、不具合が起きたとき「何が起きているか」を教えてくれる、頼れる味方です。
- とくに「**Console（コンソール）**」**タブ**にはエラーが赤字で出ます。困ったら、まずここを見る習慣をつけましょう。

くわしい使い方は `03_Web基礎_HTML_CSS.md` で説明します。今は「F12で開く調査画面がある」とだけ知っておけば十分です。

---

## 7. この資料の歩き方

---

- 章は **番号順** に読むのがおすすめです（前の章の知識を使います）。
  - **分からない言葉が出たら**、すぐ `10_用語集.md` を開いてください。やさしい言葉とたとえ話で説明しています。
  - **コードのサンプルには、ほぼ全行にコメント**（`#` や `//` で始まる説明）を付けています。コメントは「人間向けの注釈」で、プログラムの動作には影響しません。まずコメントだけ追って流れをつかみ、慣れたらコードも読む、という順がおすすめです。
  - **一度で理解できなくて当然** です。2回目、3回目で「あ、こういうことか」と分かってきます。
- 

## 8. つまずいたときの心構え

---

- **エラーは敵ではありません**。「どこが・どう良くないか」を教えてくれるメッセージです。怖がらず読みましょう。
  - **エラーメッセージはそのまま検索** しましょう。同じことで悩んだ人の解決策が見つかることがほとんどです。
  - **小さく試す**。いきなり大きな変更をせず、1か所だけ変えて結果を見る、を繰り返します。
  - **こまめに保存・記録**（`02_Git_GitHub.md` で学ぶGitが、これを助けてくれます）。
  - **それでも詰まったら、遠慮なく前任者・IT担当者に聞く**。聞くことは恥ではありません。
-

## 9. この章のまとめ

---

- **ファイル**=データのまとまり、**フォルダ（ディレクトリ）**=ファイルを入れる箱、**パス**=ファイルの住所。
- **ターミナル（PowerShell）**=文字でPCに命令する画面。**コマンド**=その命令の言葉。
- まず使うコマンドは **cd**（移動）・**ls**（一覧）・**pwd**（現在地）。コピペとTabキー、↑キーが助けになる。
- Webアプリは **ブラウザとサーバーの「お願い」と「返事」** で動く。
- 困ったら **F12（開発者ツール）** と **用語集**、そして **検索**。
- 完璧を目指さず、手を動かしながら、一歩ずつ。

準備運動はこれで終わりです。次は、いよいよこのアプリの全体像を見ていきましょう →「[00\\_導入と全体像.md](#)」。